Paso 5 — Creación del custom project: qué se clona, qué tareas se crean, qué se rechaza y qué queda registrado

Revisado por dos miembros del Council (Feynman, Taleb) el 2026-10-08; correcciones aplicadas.

Fecha: 2026-10-08 · **Para:** el dueño, que va a fijar los criterios él mismo · **Alcance:** el paso 5 entero: `sqx/projects/builder.py` (con `--workflow`), `sqx/projects/stage.py`, el donante congelado, el registro, `retire.py` y `sweep.py`, `bin/workflow.sh` y las reglas duras 6, 10 y 19.

Qué es: el documento de consulta del paso 5. Es un paso de **mecánica**, no de estadística: no juzga ninguna estrategia ni tiene criterio en `pipeline/autopilot/criteria.yaml`. Su «criterio» son las negativas del código (sección 4). Por eso es corto y no lleva estadística.

Qué código describo. El commit `b2b4c8c`. Tiene cambios sin confirmar en la carpeta del paso (`git status --short sqx/projects bin/workflow.sh`): `bin/workflow.sh`; en `sqx/projects/`, `README.md`, `buildrules.py`, `live.py`, `mcretest.py`, `orders.py`, `perturbations.py`, `registryxml.py`, `resources.py`, `retire.py`, `setups.py`, `source.py`, `stages.yaml`, `tasksettings.py`, `wfm.py`, `workflow.py`; y sin seguimiento: `buildcensus.py`, `crossfeeds.py`, `exiting.py`, `sessiondb.py`, `taskcap.py`; fuera de la carpeta, `pipeline/autopilot/resume.py` también está sin seguimiento. Tres de esos cambios tocan lo que aquí se cuenta: `stages.yaml` (el MC Retest pasa de ocho tareas a tres, 2026-10-07), `workflow.py` (aparece `trim`, el recorte `--until`) y `retire.py` (negarse si hay supervivientes sin guardar en el almacén). `builder.py`, `stage.py`, `registry.py`, `sweep.py` y `databanks.py` están sin cambios respecto al commit. **Lo que describo es lo que el código hace en el árbol de hoy**, con esos cambios incluidos.

De dónde salen los números. Todos los ejemplos son de USDJPY, M30, larga, plantilla `doubleROCLong`, de dos proyectos del 2026-10-08 que leí sin escribir nada: - `Test_USDJPY_M30_doubleROCLong_10k_B5` (creado a las 01:24, archivado a las 03:09): sólo existe ya como `AlgoData/projects/retired/SQX_w2/…_B5-2026-10-08.tar.gz`. Es la **creación más reciente de un proyecto entero con sus topes reales** (10.000 estrategias, 90 minutos). Está recortado al paso 11 por `--until crosstf` y ya corrido, así que no trae las tareas de los pasos 13 a 19. - `Test_USDJPY_M30_doubleROCLong_10k` (el vivo en `SQX_w2`, tocado a las 09:24): lo reconstruyó `/store-resume` con el mismo builder (`pipeline/autopilot/resume.py:83-86`) y **sí trae las 19 tareas**. Es un proyecto **en marcha**: lo leí en una copia a las ~09:24 (UTC), sin tocar el original, y su `project.cfx` cambió después (09:37: ya hay un snapshot nuevo y están encendidas `SPP IS` y `SPP OOS`). Todo lo que digo de «la copia viva» es el estado de las 09:24. Sus topes de generación son 1 estrategia y 1 minuto, porque la reanudación no vuelve a construir.

Las carpetas `AlgoData/projects/snapshots/` de B5 están vacías; la de la viva estaba vacía a las 09:24 y hoy tiene un snapshot de las 09:37 que no leí. Por eso lo leído sale de esos dos ficheros. Los ficheros temporales (`tasks.py`, `costs.py`) están en `scratchpad/paso05/`.

Ninguna línea de este documento está congelada. Las cifras de coste (`assets/`) y los filtros de aceptación (`assets/_study.yaml`) están marcados provisionales por sus propios ficheros.

---

0. En una página

La pregunta del paso. ¿Está bien montado el proyecto donde va a vivir todo el workflow? Concretamente: un único `project.cfx` con todas las tareas, cada una con su ventana de datos, sus costes, su entrada y su salida correctas, y sin ninguna estrategia perdida por el camino.

Lo cargado hoy. No hay criterio de estrategias: sólo negativas del código (nombre, dirección, tope de condiciones, costes sin pactar, install encendido, sesión que falta, plantilla ignorada, tarea sin costear, feed del donante que sobrevive). Ninguna es estadística.

Lo que sale en la corrida real. Un proyecto de **19 tareas** y **30 databanks declarados**, con sólo `CONSTRUCCION` y `OOS` encendidas al crearlo. El `Build` se crea con 4 filtros de aceptación (`100 <= NumberOfTrades <= 750` , `ProfitFactor >= 1.0` , `NetProfit > 0` , todos sobre IS). Las otras 18 tareas se crean con **cero condiciones de aceptación activas**. Los costes del tramo `build` son spread 0,51 y deslizamiento 0,18; los de `oos1` , 0,58 y 0,20; los de `oos2` , 0,77 y 0,27 (en puntos de SQX; el spread 0,51 son 0,0051 yenes, 0,51 «pip» del yen; sección 3.3).

Las nueve cosas que conviene saber:

1. **El builder crea las 19 tareas de golpe**, no una por paso. De las 17 del donante conserva 9 (las que algún paso corre), añade 4 (`CrossTF` y las tres patas del WFC) y 6 más al final (las tres `CrossTF <TF>` y sus tres gemelas «build»). Cada paso posterior sólo **reescribe** su tarea (`workflow.py:75-88` , `:140-167`).
2. **Al crearlo, el builder pone 15 de las 18 tareas no-Build al precio y en la ventana de `oos1`** , sea cual sea el paso que las vaya a usar (`configure.py:26-27` , `:104-142`); las otras tres, las gemelas «CrossTF build», nacen en `build` con costes de `build` porque las copia así `wf.finish` (`workflow.py:151-153` ; visto en B5). Quien las pasa a `build` u `oos2` es el configurador de cada paso, después. En la copia viva de las 09:24 se veía: `MCR 1 Bar` y `MCR 7 OHLC` ya en 2008-2017 (las reescribió el paso 13) y `WFC 3 OOS2` en 2023-2026-09-30 (la reescribió el 16.5), mientras `SPP IS` seguía en 2018-2022; a las 09:37 `SPP IS` ya está en 2008-2017 (0,51 · 0,18), reescrita al pasar el paso 15. El blanco se mueve: no tomes esas ventanas «luego» como fijas.
3. **Todas las tareas no-Build se crean «mudas»**: sin condiciones de aceptación activas y sin borrar lo que falla (`builder.py:99-101` , `crosschecks.py:62-78`). Comprobado en las 18 tareas de la copia viva: cero condiciones activas y `DeleteFailedStrategies` falso.
4. **Un proyecto con un nombre que ya existe no se rechaza en el builder**, sólo en `bin/workflow.sh:57` . El builder sobrescribe el `project.cfx` , deja los `databanks/` del anterior y borra la fila viva del registro (sección 5, punto 2).
5. **El builder escribe en el install antes de pasar sus puertas**: copia la plantilla a `…/StrategyTemplates/authored/` en la línea 91 —**truncando y reescribiendo** la que otro proyecto vivo puede estar usando—, y sus rechazos más tardíos llegan después. **Con `--workflow` sólo se pueden construir M30 y H1**: M15 muere con un mensaje, H4 pasa las puertas y revienta con `KeyError 'H4'` (sección 5, puntos 1 y 4).
6. **El filtro de aceptación del `Build` pide 10 operaciones por año, no 35.** Es deliberado y está escrito en `_study.yaml` ; el suelo de 35 vive en Python (paso 8).
7. **Un proyecto «simple» no se puede reanudar**: el registro guarda `simple:long` donde va la ruta de la plantilla, `resume.py:85` se lo pasa a `--template` y `builder.py:209` lo rechaza (sección 5, punto 3).

8. **Ninguna puerta comprueba que la ventana escrita en cada tarea sea la de su tramo**, y `registryxml.clip` recorta sin avisar el rango de datos de cada símbolo (sección 5, punto 5).
9. **`sweep` retira a las 24 h un `Test_` parado entre pasos y borra sus databanks**; la única guarda es la de supervivientes del paso 8 (sección 5, punto 8).

---

1. Qué es el paso y cómo decide

1.1 Dónde está

Es el paso 5 de la secuencia de `docs/AgentPDFs/WORKFLOW.md`. Lo ejecuta una sola orden:

```
python3 -m sqx.projects.builder <Test_|Trade_|Research_ nombre> --purpose "<para qué>"
    (--template <.sqx> | --simple long|short) --symbol <SÍMBOLO> --role custodian
    --timeframe <TF> --workflow [--max-strategies N --minutes M] [--random]
```

En la práctica casi nadie la escribe: la lanza `bin/workflow.sh simple|template` (regla 19), que antes pasa `python3 -m core.assets <SÍMBOLO>` (regla 5), se niega si el proyecto ya existe (`[[ -e $CFX ]] && die`), siempre añade `--workflow`, y después aplica la paleta (paso 6). Con `--until PASO` recorta el proyecto (sección 3.8).

El paso **decide una sola cosa por sí mismo**: si el proyecto se puede crear o se rechaza. No filtra estrategias.

1.2 Las reglas duras que lo gobiernan

- **Regla 6.** El nombre es `Test_…`, `Trade_…` o `Research_…`, sólo letras, cifras y guiones bajos (`registry.py:18` : `^(Test|Trade|Research)_[A-Za-z0-9_]+$`). El motivo del guion bajo: la API HTTP de SQX parte la orden por espacios. `bin/workflow.sh` repite la misma expresión.
- **Regla 10.** Toda corrida de SQX vive en un proyecto propio, clonado del donante congelado; nunca en el `Builder` o el `Retester` de serie. Cada tarea lleva sus propios costes. **Un workflow vive en UN proyecto.**
- **Regla 19.** El workflow se lanza con `bin/workflow.sh`, no a mano paso a paso.
- Además: regla 3 (el worker, no el master), regla 4 (no tocar un `project.cfx` que una instancia tiene abierto), regla 5 (costes pactados), regla 13 (una dirección), regla 14 (≤ 2 condiciones de entrada y de salida), regla 12 (todo en `master`, el dueño hace el commit).

---

2. Fundamentos: lo que hay que entender de SQX para leer el resto

Es corta: aquí no hay estadística.

`project.cfx`. Un fichero `.zip`. Dentro: un `config.xml` (la lista de tareas y de databanks) y un XML por tarea (`Build-Task3.xml` , `Retest-Task1.xml` …). Editar el proyecto es reescribir ese zip con el install apagado: SQX lo reescribe al guardar y al salir, y un cambio hecho con el install encendido se pierde sin avisar (regla 4). `cfxio.write` lo escribe a un fichero vecino y lo mueve encima (`cfxio.py`), y `cfxio.held` se niega si el puerto del install contesta o hay un proceso corriendo desde él.

Tarea. Una línea `<Task type=… title=… active=… taskXMLFile=…>` en `config.xml`. El `type` es `Build`, `Retest`, `ClearDatabanks` o `GoToTask`. El `title` es el nombre que muestra la ventana de SQX y que **leen todos los configuradores y cosechas**: renombrar una tarea rompe la cadena. `action=start` corre todas las tareas con `active="true"` y se salta las demás.

Databank. Una lista de estrategias con nombre. Cada tarea tiene uno de entrada y uno de salida. Ejemplo de juguete: `OOS` lee el databank `Results` (lo que generó el `Build`) y escribe el databank `OOS`; si la tarea siguiente lee `OOS`, ve sólo lo que `OOS` escribió.

Cadena. El donante trae cada tarea leyendo la salida de la anterior. Si se quitan tareas, las que quedan siguen apuntando a donde apuntaban, y una de ellas puede acabar leyendo su propia salida y no probar nada (medido el 2026-09-23, `databanks.py:46-58`). Por eso el builder re-encadena con `chain_databanks`.

Tramo. `build` (2008-2017 en USDJPY), `oos1` (2018-2022), `oos2` (2023 a 2026-09-30). Los define `assets/_policy.yaml` y las fechas salen de la ficha del activo. Cada tramo tiene su spread, su deslizamiento y su swap; por eso cada tarea lleva sus costes en sus `<Setup>`.

`<Setup>`. El bloque de una tarea con fechas, deslizamiento, spread del gráfico, método de comisión y swap. Una tarea puede llevar varios (la de la copia viva lleva dos sobre USDJPY). El builder sólo reescribe los que cotizan el activo del proyecto (`setups.py:set_costs`); uno que cotice otro mercado se deja como está.

Silenciar. Apagar todas las condiciones de aceptación de una tarea y poner `DeleteFailedStrategies` a falso. Una tarea con una condición viva borra de su salida lo que no la cumple, y entonces sólo se puede decir cuántas sobrevivieron, no cuál prueba mató a cuál. Aquí, el veredicto lo da Python.

Contar condiciones (regla 14), ejemplo hecho a mano. `doubleROCLong` tiene una condición de entrada fija (el doble ROC cuenta como una sola, decisión del 2026-10-05) y un hueco aleatorio. Tope de entrada = 2. El hueco puede llevar `2 − 1 = 1` condición, así que «What to build» de entrada se escribe **0..1**, y el total nunca pasa de 2. Si la plantilla trajera 2 condiciones fijas, no cabe hueco. Dos huecos en un lado son 0..4 condiciones y se rechazan.

---

3. Lo que hace el builder, pieza a pieza

Orden real de `builder.main()` (`builder.py:176-246`) y de `build()` (`:45-173`).

3.1 Antes de construir (en `main`)

Se ejecutan en este orden y la primera que falla corta:

#	comprobación	dónde	qué dice
1	el nombre	`registry.check_name` · `builder.py:207`	prefijo y caracteres (sección 1.2)
2	que la plantilla exista	`:209`	`--template` apunta a un fichero real
3	una sola dirección	`directions()` · `:34-42`, `:213-218`	lee `strategy_Portfolio.xml` de la plantilla y cuenta las reglas `Long entry` / `Short entry` que contienen una acción `Enter…` . Si no es exactamente una, se rechaza (regla 13). Sólo se exige si el proyecto construye (`--workflow` o `Build` en `--tasks`)
4	el tope de condiciones	`buildrules.over_the_cap` · `:219-220`	dos huecos libres en un lado, o fijas + hueco por encima del tope de 2 (regla 14)
5	costes pactados	`assetcheck.pending` · `:222-225`	un campo de coste requerido de la clase del activo con `use: null` , o una comisión con algún tramo sin valor (regla 5)
6	que ningún install lo tenga abierto	`cfxio.held` · `:226-229`	con el worker encendido no se construye en él; hay que pararlo (regla 4)
7	sesión y timeframe	`doctrine.blockers` · `:230-232`	falta `session:` en la ficha del activo, o el timeframe no está en `TF_MINUTES` (`M5, M15, M30, H1, H2, H4, D1`)
8	de dónde se toma la sesión	`source.pick` · `:233` , `source.py:30-51`	devuelve «ninguno» sólo si el donante define **ambos**, feed y sesión; si le falta cualquiera, busca el `project.cfx` más reciente de los tres installs que defina los dos y, si no hay, el más reciente que defina solo el feed (la sesión sale entonces de la base de datos de SQX). Sólo los lee; si ninguno define el feed, no se inventa

3.2 Qué se clona y de dónde

`DONOR = projects_backup("XAUUSD_base_2026-09-21")/project.cfx` (`builder.py:29`). Es una copia byte a byte del XAUUSD del master tal como estaba el 2026-09-21 (MD5 `3f997754d0465615ce329c79bb0073f4` ; lo comprobé: coincide con el `LEEME.md`). Es un punto fijo porque el XAUUSD vivo del master no lo es (SQX lo reescribe). **No se clona del vivo.** Ojo: el código **no comprueba** que el donante sea ese fichero: no hay hash en `sqx/projects/` ni en `core/datapaths.py` , `DONOR` es solo una ruta (`builder.py:29`), `--donor` la sustituye y el registro no anota cuál se usó (`registry.COLUMNS`). `kept_members` elige por **título** (`workflow.py:37-48`): un donante con un título cambiado perdería una tarea sin error. El MD5 de arriba es una comprobación mía de hoy, no una puerta.

Trae **17 tareas**: 1 `Build` (`CONSTRUCCION`), 14 `Retest` (`OOS` , `Retest Markets - Family` , `MC Trades` , `MCR 1…8` , `SPP IS` , `SPP OOS` , `WFM`), un `ClearDatabanks` y un `GoToTask` . El `GoToTask` salta incondicionalmente a la tarea 1: es un bucle de generación perpetua, y el `ClearDatabanks` vacía databanks mientras se autosincronizan, que es el mecanismo de pérdida de la regla 1. **El builder no conserva ninguna de las dos** (`keep(members, {"Build","Retest"})` , `builder.py:100-102`).

Luego, en este orden:

1. **Traducir los nombres de feed** que SQX ya no tiene (el donante es anterior al cambio de feeds del 2026-10-01): `core.symbols.current` (`builder.py:97-98`).

2. **Quedarse con las tareas que algún paso corre** (`wf.kept_members`, `workflow.py:37-48`): las de `stages.yaml` + `wfc`. Se van `MC Trades`, `MCR 2 Spread`, `3 Slippage`, `5 Params`, `6 Exits`, `8 Stress` y las dos de control. Quedan 9: `CONSTRUCCION`, `OOS`, `Retest Markets - Family`, `MCR 1 Bar`, `MCR 4 MinDist`, `MCR 7 OHLC`, `SPP IS`, `SPP OOS`, `WFM`.
3. **Añadir 4 tareas** (`wf.complete`, `workflow.py:75-88`): una copia de `OOS` titulada `CrossTF` y tres más, `WFC 1 IS`, `WFC 2 OOS1`, `WFC 3 OOS2` (los títulos salen de `wfc.tasks` en `_build.yaml:370-373`). Nacen apagadas.
4. **Pedir prestados sesión y símbolo** (`borrow_session`, `borrow_symbol`): si el donante no tiene el feed del activo (`USDJPY_M1`), se sustituye el feed del donante (`XAUUSD_M1`) por el del activo en todas las tareas.
5. **Poner el nombre** en `config.xml` y **dejar los databanks escribiendo a disco** (`sync_databanks`, `patch.py:101-114`: el donante trae los de salida en «Auto-sync never», que construye pero deja la carpeta vacía; pasan a «Auto-sync every 1 hour»).
6. En **cada tarea**: apuntar la plantilla o poner la Simple Strategy (`set_template` / `set_simple`, `patch.py:32-67`), poner la generación y los topes (`set_generation`, `set_caps`), y silenciar si es un `Retest` (3.5).
7. **Re-encadenar los databanks** (`chain_databanks`) y, con `--workflow`, **recablear** las tareas añadidas (`wf.rewire`, 3.4).
8. **Montar el zip fuera del install** (en `tmp_dir()`) y sólo ahí aplicar `configure` (costes y ventana por tarea), `rankings.apply` (filtros del `Build`) y `wf.finish` (las seis tareas finales). Se instala moviéndolo a `install/user/projects/<nombre>/project.cfx` cuando `refuse()` ha pasado (`builder.py:139-172`).

3.3 Qué se escribe en cada tarea: lo común

`configure()` (`configure.py:114-134`) recorre todas las tareas. Para cada una, `apply()` hace:

- **Costes y ventana por tarea** (`setups.set_costs`): fechas, deslizamiento, spread del gráfico, método de comisión y swap en cada `<Setup>` del activo, más el rango de datos que carga (`set_data_range`, recortado a las barras que el registro de SQX tiene) y el rango `OutOfSample` (el del donante era el de XAUUSD).
- **Doctrina de `_build.yaml`** (`doctrine.apply_doctrine`): el **mismo timeframe** en todos los gráficos de todas las tareas, precisión 1 en todas, motor `MetaTrader5 (hedged)`, sesión del activo, sizing por riesgo ATR (`Amount 1000`, `ATRMult 4`), salida el viernes a las 21:00 y tope del databank. Sólo el `Build` lleva generador y, en él: tipos de orden (sólo `EnterAtMarket`), tipos de salida, complejidad, sin SL/PT.
- **Cross-checks** (`set_crosschecks`): en el `Build` queda encendido sólo `MonteCarloManipulation`; en las demás no queda ninguno de los de la doctrina (`default: []`).

Qué tramo y qué precio le pone el builder a cada tipo (`configure.py:25-26`, `BY_TASK = {"Build": "build"}`, resto `oos1`):

tarea	tramo que deja el builder	spread · deslizamiento (USDJPY)
`Build`	`build` 2008-01-01..2017-12-31	0,51 · 0,18
cualquier `Retest`	`oos1` 2018-01-01..2022-12-31	0,58 · 0,20

En **todos** los tramos de USDJPY: comisión `SizeBased` 5,0 (el método `PercentageBased` queda apagado), swap en puntos `+2,57` largas / `−22,15` cortas, triple el miércoles, cierre a las 23:00 (leído de `core.assetdata.sqx_settings` y de las tareas). **Unidades** (resueltas por el revisor con `assets/_classes.yaml`, `knowhow/costs/mt5-points-to-sqx-points.md` y `commission-methods.md`; yo no las probé contra SQX): el spread y el deslizamiento van en **puntos de SQX** = el tamaño del tick del instrumento, que en USDJPY es 0,01 (`assets/symbols/USDJPY.yaml:24`); 0,51 puntos = 0,0051 yenes = 0,51 «pip» del yen = 5,1 puntos de MT5. La comisión `SizeBased 5,0` son **5 USD por lote, ida y vuelta, cobrados una vez al abrir**. El swap va en puntos de SQX por noche.

3.4 Las 19 tareas, en orden, con su paso, su entrada y su salida

Leído de la copia viva (`Test_USDJPY_M30_doubleROCLong_10k`) y contrastado con el código. «Ventana al crear» es la que deja **el builder**; «luego» es lo que escribe el configurador del paso.

#	título (`Retest-TaskN` / `Build-Task3`)	paso	lee → escribe	ventana al crear · luego	encendida al crear
1	`CONSTRUCCION` (Build-Task3)	6	`Results-Rexpect` → `Results`	`build`	sí
2	`OOS` (Retest-Task1)	7	`Results` → `OOS`	`oos1`	sí
3	`Retest Markets - Family` (Task3)	9	`OOS` → `Retest Markets - Family`	`oos1` · luego `build..oos1`, cada mercado con su ficha (el Setup del activo no lo muestra; ver 5.8)	no
4	`MCR 1 Bar` (Task5)	13	`Retest Markets - Family` (cadena) → `CrossTF_Mothers` (recableado) → `MCR 1 Bar`	`oos1` · luego `build` (visto en la copia viva)	no
5	`MCR 4 MinDist` (Task8)	13	`CrossTF_Mothers` → `MCR 4 MinDist`	`oos1` · luego `build` (sin configurar en la copia viva)	no
6	`MCR 7 OHLC` (Task11)	13	`CrossTF_Mothers` → `MCR 7 OHLC`	`oos1` · luego `build` (visto)	no
7	`SPP IS` (Task13)	15	`MCR 7 OHLC` → `SPP IS`	`oos1` · luego `build` (visto a las 09:37)	no
8	`SPP OOS` (Task14)	15	`SPP IS` → `SPP OOS`	`oos1`	no
9	`WFM` (Task2)	19	`SPP OOS` → `WFM`	`oos1` · luego `build..oos2` con costes de `oos1` (el WFM fija sus fechas a mano sobre el precio de `oos1`, `wfm.py:120-134`; el resto de ventanas largas se cobra al último tramo, `setups.py:span`)	no
10	`CrossTF` (Task15)	11	`CrossTF_Input` → `CrossTF`	`oos1` (`wf.finish` lo fija al tramo de `crosstf.segment`, también `oos1`)	no
11	`WFC 1 IS` (Task16)	16.5	`WFC_Variants` → `WFC_Build`	`oos1` · luego `build`	no
12	`WFC 2 OOS1` (Task17)	16.5	`WFC_Variants` → `WFC_OOS1`	`oos1`	no
13	`WFC 3 OOS2` (Task18)	16.5	`WFC_Variants` → `WFC_OOS2`	`oos1` · luego `oos2` (visto: 2023-01-01..2026-09-30, 0,77 · 0,27)	no
14	`CrossTF M15` (Task19)	11	`CrossTF_Input` → `CrossTF_M15`	`oos1`, gráfico en M15	no
15	`CrossTF H1` (Task20)	11	`CrossTF_Input` → `CrossTF_H1`	`oos1`, H1	no
16	`CrossTF H4` (Task21)	11	`CrossTF_Input` → `CrossTF_H4`	`oos1`, H4	no
17	`CrossTF M15 build` (Task22)	11.7	`CrossTF_Input` → `CrossTF_M15_build`	`build` (0,51 · 0,18)	no

#	título (**`Retest-TaskN`** / **`Build-Task3`**)	paso	lee → escribe	ventana al crear · luego	encendida al crear
18	`CrossTF H1 build` (Task23)	11.7	`CrossTF_Input` → `CrossTF_H1_build`	`build`	no
19	`CrossTF H4 build` (Task24)	11.7	`CrossTF_Input` → `CrossTF_H4_build`	`build`	no

Cómo se llega a esas entradas y salidas:

- **El orden de las filas 1-9** es el del donante. `chain_databanks` hace que cada tarea lea la salida de la anterior en el orden de `config.xml` (`databanks.py:38-71`); la primera conserva su entrada (una generación genética no lee su `Input` , parte de los databanks del sistema).
- **Las filas 4-6 y 10-13 las recablea `wf.rewire`** (`workflow.py:91-112`) **después** de la cadena, porque «cada una lee lo de la anterior» es falso para ellas: `CrossTF` lee los supervivientes cargados en `CrossTF_Input` (paso 10.5); las tres tareas MC Retest leen las mismas madres, `CrossTF_Mothers` , y **no se encadenan entre sí**; las tres patas del WFC leen el mismo lote de variantes. Una consecuencia buscada: `SPP IS` queda leyendo la tarea **anterior** en el orden, `MCR 7 OHLC` , y por eso `stages.yaml` la deja la última del MC Retest (el paso 14 corta ese databank).
- **Las filas 14-19** las crea `wf.finish` (`workflow.py:140-167`), copiando `CrossTF` ya costeada, con el timeframe de cada hermana (los de `crosstf.timeframes` de `_build.yaml:172-174` : para madres M30, `M15, H1, H4` ; para H1, `M30, H4, D1` , D1 con motor `MetaTrader4`). Las gemelas «build» son las del paso 11.7, sin D1.
- **Los databanks declarados son 30**: 21 del donante, menos 4 que `keep` descarta, más 13 que declara `declare()` (`wfc.py:122-141`). `keep` conserva todo databank que un `Input` / `Output` de una tarea conservada aún nombre *antes* de re-encadenar (`sqx/inspect/keep_tasks.py:49-56`): `MCR 4 MinDist` leía `MCR 3 Slippage` , `MCR 7 OHLC` leía `MCR 6 Exits` y `SPP IS` leía `MCR 8 Stress` , por eso esos tres quedan declarados sin tarea que los escriba; `MCR 2` y `MCR 5` solo los leían tareas descartadas.

3.5 Lo que se silencia y lo que no

`--workflow` fuerza `silence = {"Retest"}` (`builder.py:99-101`). Las 18 tareas de tipo `Retest` pierden sus condiciones de aceptación (el `Build` es la decimonovena). **El `Build` conserva las suyas**, y son las que escribe `rankings.apply` desde `assets/_study.yaml` , sustituyendo ENTERAS las del donante (`NumberOfTrades > 500` , `SortinoRatio > 0.6` , `Stagnation < 540`).

Ejemplo real, `Build` de USDJPY M30 (B5):

condición	valor	de dónde
`NumberOfTrades >=`	100	`ceil(10 por año × 10 años)`
`NumberOfTrades <=`	750	`floor(75 por año × 10 años)`
`ProfitFactor >=`	1,0	`profit_factor_min`
`NetProfit >`	0	`net_profit_min`

Los «10 años» son `round((2018-01-01 − 2008-01-01) / 365,25 días, 2) = 10,0` (`core/buildfilters.py:64`; el tramo `build` acaba el 1 de enero de 2018 a las 00:00, no el 31 de diciembre, y el cociente sin redondear da 10,0014, medido). Sin el redondeo **el suelo** saldría `ceil(100,014) = 101` y el techo seguiría en 750. Todas se leen sobre IS (`sample: IS`, el único admitido: «seleccionar con el OOS lo gasta»). Los valores por timeframe son `H1` / `M30` 10-75 y `H4` 10-100 por año; un timeframe que no está en `_study.yaml` no se puede construir.

3.6 El `Build`: lo que la doctrina le escribe

Leído de `Build-Task3.xml` de B5:

- **Plantilla:** `StrategyType type="template"` y `templateFile` apuntando a `~/Desktop/SQX_w2/user/settings/StrategyTemplates/authored/doubleROCLong.sqx`. El builder fuerza `type="template"` porque con `type="simple"` SQX construiría genérico e ignoraría la plantilla sin ningún error (`patch.py:32-45`). Una Simple Strategy usa en su lugar la plantilla de serie `SQ3StrategyTemplateExample.sq4` y el lado en `MarketSides` (`patch.py:48-66`).
- **Dirección:** `MarketSides type="long"`, simetrías de entrada y salida en falso.
- **«What to build»:** entrada 0..1 (una condición fija más un hueco: 2 − 1) y salida 0..2 (la plantilla no fija **ninguna** condición de salida, así que cabe el tope entero: 2 − 0; medido con `buildrules.conditions`: entrada fija 1 y hueco 1, salida fija 0). La búsqueda del «0» de la entrada es del propio código (`what_to_build`, `buildrules.py:116-141`): con una fija, el mínimo es 0; si el hueco está atado a un grupo, se escribe 1..1.
- **Órdenes:** sólo `EnterAtMarket` encendido. **Salidas:** salida por barras obligatoria con rango en horas convertido a barras: `2 h` a `24 h` en M30 son `4..48` barras (`bar_range`, `buildrules.py:66-79`); la salida por condición es optativa; sin SL, PT, trailing ni break-even.
- **Generación:** `genetic-evolution` (o `random-generation` con `--random`), con el bloque genético copiado verbatim de `buildmode_model.xml`.
- **Topes:** `MaxStrategies 10000` y `time-limit 1 h 30 min` en B5 (los pasó `workflow.sh`). Sin argumentos manda el prefijo (`doctrine.caps`, `_build.yaml:81-84`): `Trade_` y `Research_` 10.000 y 180 min; `Test_` 300 y 10 min.
- **Cross-check:** `MonteCarloManipulation use="true"`, el de la doctrina `crosschecks.build`. `RetestOnAdditionalMarkets use="false"`.

3.7 Cómo cada paso enciende sólo su tarea

`action=start` corre todas las tareas con `active="true"`. Para que cada paso corra lo suyo y nada más:

- **`stages.yaml`** dice qué títulos corre cada paso: `build: [CONSTRUCCION]`, `oos: [OOS]`, `crossmarket: [Retest Markets - Family]`, `crosstf: [CrossTF]`, `mcretest: [MCR 1 Bar, MCR 4 MinDist, MCR 7 OHLC]`, `spp: [SPP IS, SPP OOS]`, `wfm: [WFM]`; `wfc` sale de `_build.yaml` (`stage.titles`, `stage.py:30-41`).
- **`stage.only`** (`stage.py:44-66`) reescribe el `active=` de cada `<Task/>` por expresión regular: encendida si su título está en la lista, apagada si no. **Una tarea sin título (un `ClearDatabanks` o un `GoToTask` de un donante) se deja siempre apagada**: un `GoToTask` activo daría vueltas para siempre.
- **`stage.apply` / `stage.just`** (`:69-110`) lo aplican al `project.cfx` (rechazan si un install lo tiene abierto, `:97-100`) y **fallan** si el proyecto no lleva alguna de las tareas pedidas («un proyecto de workflow se crea con `builder --workflow`»), porque `start` correría nada o lo equivocado.

- **`stage.own`** (`:113-133`) es lo que llama cada configurador al terminar (`crossmarket` , `crosstf` , `mcretest` , `spp` , `wfm` , `wfc`): deja encendido sólo su paso. En un proyecto que no es de workflow (por ejemplo, uno recortado) devuelve un aviso y no cambia nada.
- **Excepciones:** `crosstf.py:115-116` enciende `CrossTF` más las hermanas (`just`), y las gemelas «build» del 11.7 las enciende el lanzador de la ventana (`ui/daemon/launch/steps.py:105`); `crosstfload.py:158-163` solo las lista (`phase2_tasks`).
- **Al terminar el builder** `wf.finish` llama `apply(cfx, ["build", "oos"])` (`workflow.py:167`): sólo `CONSTRUCCION` y `OOS` quedan encendidas. Comprobado en B5 (`CONSTRUCCION` y `OOS` activas, el resto no).

En la copia viva a las 09:24 estaban encendidas `MCR 1 Bar` y `MCR 7 OHLC` (paso 13, sólo el control y la historia, decisión del 2026-10-07 en `stages.yaml`); a las 09:37, `SPP IS` y `SPP OOS` .

Consecuencia de `stage.own` en un proyecto recortado (Taleb, comprobado en `stage.py:127-130`): exige que estén los títulos de **todos** los pasos de `stages.yaml` ; en un proyecto cortado con `trim` faltan `MCR…` , `SPP…` , `WFM` , así que devuelve el aviso y **no cambia ninguna tarea** —ni siquiera la de `crossmarket` —. `CONSTRUCCION` y `OOS` siguen encendidas y un `action=start` a mano relanzaría el Build. Lo que de verdad deja encendida solo la tarea del paso es la llamada a `stage.apply` que hace el lanzador justo antes de `start` (`ui/daemon/advance/run.py:206` , `ui/daemon/launch/run.py:163`), según Taleb; yo no abrí esos dos ficheros.

3.8 El recorte `--until` (nuevo, sin confirmar)

`bin/workflow.sh simple|template … --until oos|crossmarket|crosstf|mcretest|spp` llama `workflow.trim` justo después del builder. Borra del `project.cfx` las tareas (y sus XML) de los pasos posteriores; sus databanks quedan declarados y vacíos (`workflow.py:115-137`). B5 se cortó así en `crosstf` : de sus 10 tareas quedan `CONSTRUCCION` , `OOS` , `Retest Markets - Family` , `CrossTF` y las seis hermanas. **`trim` no quita las hermanas ni las gemelas «build»**, porque no figuran en `stages.yaml` (sección 5, punto 6).

3.9 Los rechazos de dentro de `build()`

Salen después de las puertas de `main()` :

rechazo	dónde	qué lo dispara
falta la sesión	`doctrine.borrow_session` · `:84-124`	ni un proyecto ni la base de datos de SQX (`sessiondb`) definen la sesión: «no se inventan horarios»
ninguna tarea define la sesión	`configure.py:136-142`	tras todo, `unify_sessions` no encuentra la definición en ninguna tarea
sin filtro para el activo/timeframe/modo	`buildfilters.py` · `_need`	falta un valor en `_study.yaml` (por ejemplo, un timeframe que no está) o `sample` distinto de `IS`
plantilla ignorada	`resources.refuse` · `:143-144`	una tarea nombra una plantilla pero declara `type="simple"` (OPEN.md #9: 0 de 642)
tarea sin costear	`refuse` · `:145-149`	una tarea sin ningún `<Setup>` sobre el feed del activo: operaría el mercado del donante a sus costes
feed del donante vivo	`refuse` · `:150-156`	tras la sustitución de feed, `XAUUSD_M1` aparece todavía en algún XML

Los rechazos de `refuse` ocurren antes de mover el zip al install: no queda un `.cfx` a medias en `user/projects` (OPEN.md #34, el build de USDJPY que cargó oro).

3.10 Qué queda registrado

`registry.record(done, purpose, symbol, template)` (`builder.py:242`, `registry.py:87-106`) añade una fila a `AlgoData/projects/registry.csv`, bajo un candado (`fcntl.flock` sobre `registry.csv.lock`, `registry.py:63`) y escribiendo a un fichero vecino que luego se mueve encima (`:81`). Las 12 columnas: `name, kind, install, created, purpose, symbol, timeframe, template, workflow, retired, archive, acceptance`.

Ejemplo real (la fila de B5, resumida): `kind` `test`; `install` `SQX_w2`; `created` `2026-10-08 01:24` (UTC); `purpose` «Lote 5 de la misma celda … estudio de lotes de la noche del 2026-10-07, cortado tras el gate del paso 8»; `symbol` `USDJPY`; `timeframe` `M30`; `template` la ruta de `doubleROCLong/template.sqx`; `workflow` `yes` (porque `done["added"]` no está vacío; **un proyecto recortado con `--until` también consta como `workflow=yes`**: el recorte ocurre fuera del builder, `bin/workflow.sh:62-64`, y no se anota); `retired` `2026-10-08 03:09`; `archive` la ruta del `.tar.gz`; `acceptance` `study @IS: NumberOfTrades >= 100 & NumberOfTrades <= 750 & ProfitFactor >= 1.0 & NetProfit > 0`.

Estado del registro hoy (leído por mí): **161 filas** (139 `test`, 21 `legacy` anteriores a los prefijos, 1 `trade`; ninguna `research`), 150 retiradas y 11 vivas. De las filas, 54 tienen `workflow=yes`, 85 `no` y 22 vacío (anteriores a esa columna).

Además del registro, el builder deja: el proyecto en `<install>/user/projects/<nombre>/project.cfx`, la plantilla en `<install>/user/settings/StrategyTemplates/authored/<carpeta>.sqx` (el nombre es el de la carpeta de la biblioteca, porque todas se llaman `template.sqx` y dos se pisarían sin avisar, `builder.py:84-91`), **La copia de la plantilla se sobrescribe cada vez que se construye un proyecto con ella**: los proyectos vivos del mismo install que la apuntan pasan a leer la versión nueva. Además deja por pantalla un resumen (`summary.say`) con las advertencias al final: costes provisionales, aceptación silenciada. Con `--json`, sólo el JSON. El resultado entero **no se guarda en ningún sitio**: sólo lo que va a la fila.

3.11 Retirar y barrer

- **`retire`** (`retire.py:55-104`): `python3 -m sqx.projects.retire <P> --role <rol> --yes`. Se niega si es un proyecto de serie (`Builder`, `Retester`, `Optimizer`, `PortfolioComposer`, `PortfolioMaster`), si no hay `project.cfx`, si el install lo tiene abierto, si el zip es ilegible, si cambió hace menos de 60 minutos (alguien puede estar cosechando; `--own` lo salta), si falta un databank pedido con `--keep`, o (nuevo, sin confirmar) si hay supervivientes del paso 8 que el almacén no tiene enteros. Archiva **sólo el** **`project.cfx`** (más los databanks de `--keep`) en `AlgoData/projects/retired/<install>/<nombre>-<fecha>.tar.gz`, borra la carpeta del install y marca la fila del registro como retirada con la fecha y la ruta del archivo. Sin `--yes` es un ensayo.
- **`sweep`** (`sweep.py:35-81`, lunes 02:50 por cron, vía el agente `projectJanitor`): recorre los tres installs y para cada proyecto decide, en este orden: install encendido → se queda; zip ilegible → se queda; tocado hace menos de 24 h → se queda; en la cola `retire-queue.txt` → se retira; `Trade_` → se queda; `Research_` → se queda; `Test_` → se retira; menos de 10 tareas → se retira; si no, se queda («workflow entero»). **Del master sólo se mira lo que el dueño puso en la cola** (`:55-56`).

Lo que no distingue (Taleb, comprobado en `sweep.py` y `retire.py:114`): un `Test_` terminado de uno **parado entre dos pasos**. Lo retira a las 24 h de inactividad con el install apagado, y `sweep_all` llama a `retire(…, [], True)` sin `--keep`, así que se borran los databanks de todos los pasos y solo se archiva el `project.cfx`. La negativa por supervivientes sin guardar mira únicamente las del paso 8 (`retire.py:88-92`); lo que producen CrossTF, MC Retest, SPP, WFC y WFM en databanks no lo guarda nada.

---

4. El criterio cargado y por qué

No hay reglas de estrategias. Lo que juzga el paso son las negativas de 3.1 y 3.9, cada una con su motivo:

negativa	motivo	quién la mandó
prefijo y caracteres del nombre	distinguir trabajo de basura; la API parte por espacios	regla 6, 2026-09-26 / 2026-10-03
una dirección por plantilla	dos direcciones en una estrategia mezclan dos ideas	regla 13, 2026-10-01
≤ 2 condiciones de entrada y de salida, huecos contados por lado	evitar sobreajuste y 0..4 condiciones sin querer	regla 14, 2026-10-02
costes sin pactar	un resultado con un spread inventado arrastra toda la corrida	regla 5
install encendido	SQX reescribiría el `.cfx` al salir y se perdería el cambio	regla 4
sesión que falta	una tarea que nombra una sesión que no define opera en horas equivocadas sin error	OPEN.md
plantilla ignorada	SQX construye genérico sin avisar	OPEN.md #9
tarea sin `<Setup>` del activo / feed del donante vivo	el proyecto operaría oro con los costes del oro	OPEN.md #34-35
aceptación del `Build` sólo sobre IS	seleccionar con el OOS lo gasta	`_study.yaml`
todo `Retest` silenciado	una tarea que borra lo que falla convierte la medida en selección	owner, 2026-09-24/25

`oos2` : el builder no lo escribe en ninguna tarea al crear (su `--segment` solo admite `build|oos1` , `builder.py:193`). Pero la negativa a `oos2` de `setups.span` y `configure.main` solo muerde con `ALGO_AUTONOMOUS=1` , y hoy no lo pone nadie (`core/assetdata.py:18-27`): la protección es de documento, no de código.

Dos decisiones del dueño que el builder cumple y que no son una negativa: que las tareas **no se re-encadenen** donde la cadena es falsa (3.4) y que el filtro del `Build` sea **mínimo a propósito** (10 operaciones al año, no 35).

---

5. Lo que no cuadra o no está medido

Cada punto lo saqué del código o lo comprobó un revisor; **no ejecuté el builder** (la brief prohíbe tocar SQX y crear proyectos). Donde un revisor midió con una función suelta, lo digo.

1. **El builder escribe en el install antes de pasar sus puertas, y lo que escribe es compartido.** El comentario de `builder.py:134-138` dice «Nothing under `install` is touched until every gate here has passed», pero `install_template` (`builder.py:87-91`) abre el destino con `ZipFile(target, "w")` (`patch.py:18-29`): **trunca y reescribe** `authored/<carpeta>.sqx` . Ese fichero lo comparten todos los proyectos de la misma plantilla en el mismo install (B5 y la copia viva apuntan los dos a `…/SQX_w2/user/settings/StrategyTemplates/authored/doubleROCLong.sqx`). Un build rechazado, o repetido, reemplaza la plantilla bajo un proyecto vivo; reconstruir uno tras editar la plantilla de la biblioteca cambia en silencio la plantilla que otro (por ejemplo un `Trade_`) tiene en su `Build` . SQX no está encendido en ese momento (`cfxio.held` pasa antes, `builder.py:226`), así que no hay corrupción, pero el efecto es silencioso.

2. **Un nombre repetido se pisa.** `main()` no comprueba si el proyecto existe. `build()` termina en `shutil.move(staged, out)` (`builder.py:172`): sustituye el `project.cfx` y **deja intacta la carpeta `databanks/` del anterior** (el builder nunca la toca; solo `retire` hace `rmtree`, `retire.py:102`). Además `registry.record` borra la fila viva anterior del mismo nombre e install (`registry.py:102-105`): se pierden su `purpose` y su `created`. La única negativa es `bin/workflow.sh:57` (`[[ -e $CFX ]]`); no cubre `resume.py`, la llamada directa al builder ni los lanzadores de la ventana. Lo que no sé es si SQX carga esos databanks viejos al abrir el proyecto nuevo.
3. **Un proyecto «simple» no se puede reanudar.** El registro anota `simple:long` donde va la plantilla (`builder.py:242`); `resume.py:85` lo pasa a `--template`; `builder.py:209` (`Path('simple:long').exists()` es falso) lo rechaza con «simple:long does not exist» y `resume.run` lo convierte en «no se reanuda nada». Ocurre tras `core.assets` y antes de escribir nada. Hoy las 6 filas `simple:long` del registro (`Trade_NIKKEI225_M30_volatilidadL` y 5 `Test_NIKKEI225_M30_simpleL_*`) están vivas y `resume` exige una retirada, así que ninguna es reanudable aún; al retirarse, fallarán así. Un arreglo mínimo que propone el revisor: en `resume.py:85`, usar `--simple <lado>` cuando la plantilla empiece por `simple:` (el grupo es excluyente, `builder.py:181-183`); no hace falta columna nueva. Y aunque la plantilla fuese legible, `resume.py` reconstruye con los costes, tramos y filtros **de hoy**, no con los del proyecto original: el registro no guarda ni el donante, ni los costes, ni la versión de la plantilla.
4. **Timeframes: con `--workflow` solo se construyen M30 y H1.** La ayuda de `--timeframe` dice «M15, M30, H1 or H4» (`builder.py:187`); `blockers` acepta `M5, M15, M30, H1, H2, H4, D1` (`buildrules.py:11`); `_study.yaml` admite `H1`, `M30`, `H4`. Pero `crosstf.timeframes` (`_build.yaml:172-174`) solo tiene `M30` y `H1`, y `wf.finish` (`builder.py:148`) indexa por el timeframe de la madre: **H4 pasa todas las puertas y revienta con `KeyError 'H4'`** (un traceback, no un mensaje) al montar las hermanas; M15 muere antes, en `rankings.apply` → `buildfilters._need`, con un `SystemExit` limpio («falta `M15` en `trades_per_year`»). M5, H2 y D1 mueren de una de las dos formas. Ambos casos dejan la plantilla ya copiada (punto 1). Medido por el revisor con `planned('H4')` y `planned('M15')`; yo no ejecuté el builder.
5. **Ninguna puerta comprueba que la ventana de cada tarea sea la de su tramo.** `resources.refuse` solo mira tres cosas (plantilla ignorada, tarea sin `<Setup>`, feed del donante vivo; `resources.py:143-156`); `result["segments"]` se calcula (`builder.py:164`) y no lo lee ninguna puerta. Y `registryxml.clip` (`registryxml.py:45`, llamado desde `setups.set_data_range`) **recorta sin avisar** el rango de datos de cada `<Symbol>` a las barras que tenga el registro de SQX: los `<Setup>` conservan las fechas del tramo, pero SQX carga menos barras. Los filtros del `Build` usan los años **nominales** del tramo (`core/buildfilters.py:63-64`), no los cargados: un feed que empezara tarde daría un Build de menos años con el suelo y el techo de operaciones de diez.
6. **`trim` deja las seis tareas de hermanas.** Quita los títulos de los pasos posteriores definidos en `stages.yaml` y `wfc`, y `CrossTF M15`… `CrossTF H4 build` no están ahí. Con `--until oos` o `--until crossmarket` quedarían, apagadas, seis tareas de un paso que el proyecto ya no tiene. B5 (cortado en `crosstf`) no lo demuestra. El único test de `trim` es `tests/test_autopilot_crosstf_cascade.py:190-205` y solo cubre `crosstf`. Su consecuencia con `stage.own` está en 3.7.
7. **Un segundo `<Setup>` a coste cero en el `Build`.** El de la copia viva lleva dos: USDJPY y otro sobre `AUDJPY_TICK` en 2018-2022 (el `oos1` del activo, no el `build`) con comisión «None», deslizamiento 0 y swap apagado, que `set_costs` deja intacto por cotizar otro mercado (`setups.py:166-167`). **No se ha comprobado que SQX no cargue sus datos al correr el Build**; su cross-check (`RetestOnAdditionalMarkets`) está apagado.

8. **Las ventanas «luego» de 3.4 no se ven todas en una copia.** Salen de `_build.yaml`, y solo `MCR 1 Bar`, `MCR 7 OHLC`, `SPP IS` (a las 09:37) y las patas del WFC se vieron reescritas. Dos hallazgos del revisor sobre lo no dicho: `Retest Markets - Family` ya trae **al crearse** (B5) `<Setup>` de `XAGUSD` 2008-2022 y `UKOIL.cash` 2013-2022 con spread 0, deslizamiento 0 y comisión `PercentageBased`, herencia del donante que `set_costs` deja (`setups.py:166-167`) hasta que el paso 9 los reescribe; y conviven dos reglas de coste para una ventana larga: el WFM la cobra al precio de `oos1` (`wfm.py:120-134`) y la docstring de `setups.span` dice que se cobra al último tramo.
9. **Unidades:** resueltas en 3.3 por el revisor (puntos de SQX; 5 USD por lote ida y vuelta, una vez al abrir). Yo no las probé contra SQX.
10. **Comentarios desfasados, sin efecto en el comportamiento.** `sweep.py:10` dice que un workflow lleva «15 a 18 tareas»; hoy son 19 (el umbral de 10 sigue valiendo). `retire-queue.txt` dice «lunes 03:00» y el cron es 02:50 (`50 2 * * 1`).
11. **El filtro de 10 operaciones al año frente al suelo de 35 del dueño** (2026-10-02): `_study.yaml` dice que un suelo de 35 «NO va aquí» porque pierde el 81 % de las robustas en H1 y M30, y el de 35 se aplica en el paso 8. Lo dejo escrito para que la decisión sea tuya.
12. **Tests.** Hay `tests/test_registry_names.py`, `test_builder_scope.py`, `test_builder_simple.py` y `tests/test_autopilot_step13.py:70-91` (que sí llama a `wf.rewire` y comprueba entradas y salidas de MCR, `SPP IS` y `SPP OOS`). No hay test de `stage.only` propio (los otros lo simulan: `test_advance.py:159`, `test_launch.py:24`) ni de nombres repetidos.

---

6. Cierre: las decisiones que le quedan al dueño

1. **¿Rechazar en el builder un nombre que ya existe?** Gana: no se pisa un proyecto vivo ni se mezclan databanks y filas de registro de dos corridas. Cuesta: una línea, y quien quiera rehacer uno tiene que retirarlo antes (como ya obliga `workflow.sh`).
2. **¿Rechazar en `main()` los timeframes que luego no se pueden construir (M15, H4, M5, H2, D1) y mover la copia de la plantilla a después de las puertas?** Gana: un mensaje claro en vez de un `KeyError`, y un build rechazado deja de pisar la plantilla que usa otro proyecto. Cuesta: decidir si H4 debe poder construirse (entonces falta su entrada en `crosstf.timeframes`) o salir de la ayuda.
3. **¿Cómo guardar en el registro la plantilla de una Simple Strategy?** Hoy `simple:long`. Gana: `/store-resume` funciona con proyectos simples (con el arreglo de `resume.py:85`, sin columna nueva). Cuesta: tocar `resume.py`, y decidir si la reanudación debe usar los costes de hoy o los del proyecto original, que el registro no guarda.
4. **¿Comprobar la ventana escrita de cada tarea contra su tramo, y avisar cuando `clip` recorta?** Gana: un error de ventana deja de contaminar en silencio todos los juicios posteriores. Cuesta: una puerta nueva en `refuse` y decidir qué hacer cuando el feed es más corto que el tramo.
5. **¿Qué hace `sweep` con un `Test_` parado entre pasos?** Hoy lo retira a las 24 h y borra sus databanks; solo protege las supervivientes del paso 8. Gana (si se protege más): no se pierde lo que dejaron CrossTF, MC Retest, SPP, WFC y WFM. Cuesta: decidir qué cuenta como «terminado» y qué se archiva.
6. **¿Hay que anotar el donante usado y el recorte `--until` en el registro, y comprobar el hash del donante?** Gana: un proyecto con otro donante o cortado deja de pasar por un workflow entero. Cuesta: dos columnas nuevas y un hash.

7. **¿Que `trim` también quite las hermanas?** Gana: un proyecto cortado en `oos` o `crossmarket` no arrastra tareas de un paso que no tiene. Cuesta: decidir qué títulos cuentan (hoy salen de la doctrina del paso 11, no de `stages.yaml`).
8. **¿Es la ventana del paso 5 la que quieres?** Hoy 15 de las 18 tareas no-Build nacen en `oos1` y las configura después cada paso. Gana, si cada una nace en el tramo de su paso: la copia de un proyecto recién creado ya diría la verdad. Cuesta: cambiar `BY_TASK` (`configure.py:26`) y repasar a los configuradores, que hoy lo reescriben todos.
9. **¿Se mantiene `Research_` sin barrer?** El barrido lo deja vivo sin decidir (`sweep.py:71-73`). Ninguna fila `research` existe todavía, así que de momento no cuesta nada.
10. **¿Se activa de verdad la reserva de `oos2`?** El builder no lo escribe, pero la negativa del resto del código solo muerde con `ALGO_AUTONOMOUS=1`, que hoy no pone nadie.